

CRACOW UNIVERSITY OF TECHNOLOGY
(Politechnika Krakowska)

Faculty of Computer Science and Mathematics

FINAL DEVELOPMENT REPORT

Travel Planner Application «TUM»

Authors:

Mariana Pashchuk

Tuna Sumanci

Patricio Zapata

CONTENTS

1. Project Topic, Goal and Objectives.....	3
Project Topic.....	3
Project Goal	3
Main Objectives	3
2. Page Structure and User Interface (index.html).....	4
Team Logo Implementation.....	4
Search Interface.....	5
Results Section	6
Trip List.....	6
Modal Window.....	7
3. Styling and Responsiveness (style.css).....	7
CSS Variables	8
Flexbox Layout	8
CSS Grid Layout.....	9
Toast Notifications	10
Modal Window Styling.....	11
4. Application Architecture and Business Logic (app.js)	11
4.1 Geospatial Services Integration	11
4.2 Request Optimization and Caching	12
4.3 Distance Calculation Using the Haversine Formula.....	13
4.4 Dynamic Loading of Descriptions and Images.....	13
4.5 Interactive Map with Leaflet.js	14
4.6 Dynamic DOM Manipulation	14
4.7 Filtering and Sorting	15
4.8 Travel List State Management	17
4.9 Duplicate Data Protection	18
4.10 Input Validation and Error Handling	18
4.11 Unit Testing.....	19
5. Conclusions.....	21

1. Project Topic, Goal and Objectives

Project Topic: Development of a client-side web application for travel planning (Travel Planner Application).

Project Goal: The goal of the project is to create a modern Single Page Application (SPA) that allows users to search for tourist attractions, view detailed information about locations, calculate distances to places, interact with maps, and create a personal travel itinerary.

Main Objectives: During the development process, the following objectives were defined:

- implement city search using external APIs;
- integrate geospatial services for obtaining coordinates and tourist attractions;
- create an interactive interface without page reloading;
- implement a local caching mechanism for search results;
- calculate distances between geographical points using the Haversine formula;
- implement a multi-factor filtering and sorting system;
- create a modal window with an interactive map and detailed information about locations;
- implement a travel list management system;
- prevent duplicate entries in the travel list;
- design a custom team logo using SVG technology;
- develop unit tests to verify the correctness of key algorithms.

2. Page Structure and User Interface (index.html)

The **index.html** file serves as the foundation of the entire web application. It defines the page structure and contains all major user interface components. During development, semantic HTML tags were used, including:

- `<header>`
- `<main>`
- `<section>`
- `<aside>`
- `<form>`

Example of structure from our code:

```
<header class="site-header">
</header>

<main class="layout">

  <section class="panel" id="resultsPanel">
  </section>

  <aside class="panel" id="tripPanel">
  </aside>

</main>
```

The use of semantic markup improves code readability, maintainability, and compliance with modern web development standards.

Team Logo Implementation

The project logo was created using **Inline SVG technology**. Unlike traditional PNG or JPG images, SVG graphics maintain quality at any scale and require fewer browser resources. The logo consists of:

- a location marker icon;
- decorative vector elements;

- the text "TUM Travel Planner".

How it looks like:



How we implemented it:

```
<a href="#" style="text-decoration: none; display: flex; align-items: center; gap: 20px;">
  <svg width="220" height="50" viewBox="0 0 250 60"
  xmlns="http://www.w3.org/2000/svg">

    <g transform="translate(0, 2)">
      <path d="M 25 5 C 13.9 5 5 13.9 5 25 C 5 40 25 55 25 55 C 25 55 45 40 45 25
C 45 13.9 36.1 5 25 5 Z" fill="#F97316" />
      <circle cx="25" cy="22" r="10" fill="#FFFFFF" />
      <path d="M 18 26 L 22 18 L 25 22 L 28 15 L 32 26 Z" fill="#1b6cbd" />
    </g>

    <text x="55" y="35" font-family="'Segoe UI', Arial, sans-serif" font-
size="34" font-weight="900" fill="#FFFFFF" letter-spacing="1">TUM</text>

    <text x="57" y="52" font-family="'Segoe UI', Arial, sans-serif" font-
size="11" font-weight="700" fill="#6EE7B7" letter-spacing="2.5">TRAVEL
PLANNER</text>
  </svg>
</a>
```

This approach enabled the creation of a unique visual identity without relying on external graphic editing software.

Search Interface

At the top of the page, a search bar allows users to enter the name of a city. After clicking the **Search** button, the application sends requests to external APIs and displays tourist attractions found within the selected city.

Example from our code:

```
<div class="search-bar">
  <input type="text" id="searchInput" placeholder="Search a city... e.g.
Kyiv">
  <button id="searchBtn">Search</button>
</div>
```

Results Section

A dedicated **Results** section is used to display discovered locations.

Each result contains:

- place name;
- category;
- address;
- distance from the city center;
- map viewing button;
- add-to-trip button.

Example from our code:

```
<main class="layout">

  <section class="panel" id="resultsPanel">
    <h2 class="panel-title">Results</h2>
    <div id="filterControls" style="display: none; gap: 10px; margin-bottom:
15px; flex-wrap: wrap;">

      <select id="categoryFilter" onchange="filterAndSortResults()"
style="padding: 8px; border-radius: 6px; border: 1px solid var(--border);
outline: none;">
        <option value="all">All Categories</option>
        <option value="tourism">Tourism & Sights</option>
        <option value="catering">Food & Restaurants</option>
        <option value="entertainment">Entertainment</option>
      </select>

      <select id="sortFilter" onchange="filterAndSortResults()" style="padding:
8px; border-radius: 6px; border: 1px solid var(--border); outline: none;">
        <option value="default">Default order</option>
```

```

        <option value="distance">Nearest first (Distance)</option>
      </select>
    </div>
    <div id="results">
      <p class="placeholder">Search for a city to see places.</p>
    </div>
  </section>

```

Trip List

On the right side of the page, a personal travel list is displayed.

The functionality includes:

- adding places;
- removing places;
- sorting saved locations;
- creating custom entries manually.

Modal Window

A custom modal window is implemented for detailed location viewing.

When opened, it displays:

- place name;
- category;
- address;
- image;
- short description;
- Google Maps link;
- interactive Leaflet map.

Example from our code:

```

<div id="detailsModal" class="modal-overlay" style="display: none;">
  <div class="modal-content">
    <button class="close-btn" onclick="closeModal()">X</button>
  </div>
</div>

```

```

    <h2 id="modalTitle">Place Name</h2>
    <span class="category" id="modalCategory">Category</span>
    <p id="modalAddress" style="margin: 10px 0; color: var(--muted);"></p>
    <p id="modalDescription" style="margin: 10px 0; font-size: 14px; line-height: 1.4; color: var(--text);"></p>
    <a id="modalMapLink" href="#" target="_blank" style="display: inline-block; margin-bottom: 10px; color: var(--blue); font-weight: bold; text-decoration: none; font-size: 14px;"> Open in Google Maps</a>
    
    <div id="map" style="height: 300px; width: 100%; border-radius: 8px; margin-top: 15px; border: 1px solid var(--border);"></div>
  </div>
</div>

```

3. Styling and Responsiveness (style.css)

The visual design of the project is implemented using modern CSS3 features.

CSS Variables

To simplify maintenance and improve consistency, all major colors are defined as CSS custom properties.

This approach allows centralized control over the application's color palette and simplifies future design modifications.

Example from our code:

```

:root {
  --blue:      #1b6cbd;
  --blue-lt:   #E6F1FB;
  --green:     #28C840;
  --gray:      #f4f5f7;
  --border:    #dde1e7;
  --text:      #1a1a2e;
  --muted:     #6b7280;
  --radius:    10px;
}

```

Flexbox Layout

The header section is built using the Flexbox layout model. This provides:

- proper alignment of the logo and search bar;
- automatic adaptation to different screen sizes;
- flexible positioning of interface elements.

Example from our code:

```
body {  
  font-family: Georgi, serif;  
  background: var(--gray);  
  color: var(--text);  
  min-height: 100vh;  
}  
  
.site-header {  
  background: var(--blue);  
  padding: 18px 32px;  
  display: flex;  
  align-items: center;  
  gap: 4px;  
  margin-left: -4px;  
  flex-wrap: wrap;  
}
```

CSS Grid Layout

The main content area is organized using CSS Grid.

A two-column layout is implemented:

- left column – search results;
- right column – travel list.

Example from our code:

```
.layout {  
  display: grid;  
  
  grid-template-columns: 2fr 1fr;  
  
  gap: 20px;  
  
  padding: 24px 32px;  
}
```

This structure improves content organization and enhances user experience.

Toast Notifications

A notification system provides immediate feedback to users.

Notifications appear for:

- successful searches;
- adding locations;
- removing locations;
- validation errors;
- API-related problems.

Example from our code:

```
.toast {
  position: fixed;
  bottom: 28px;
  right: 28px;
  padding: 13px 20px;
  border-radius: var(--radius);
  font-family: Georgia, serif;
  font-size: 14px;
  color: #fff;
  z-index: 9999;
  box-shadow: 0 4px 16px rgba(0,0,0,0.15);
  animation: toast-in 0.25s ease;
  max-width: 320px;
  line-height: 1.5;
}
.toast.success { background: #1a7a2a; }
.toast.error    { background: #b91c1c; }
.toast.info     { background: var(--blue); }

.toast-hide { animation: toast-out 0.35s ease forwards; }
```

Custom CSS keyframe animations are used for smooth appearance and disappearance effects.

Modal Window Styling

A separate set of styles is responsible for modal window presentation. The design includes:

- background overlay;
- fade-in animation;
- shadows;
- smooth transitions.

Example from our code:

```
.modal-overlay {
  position: fixed;
  top: 0; left: 0; width: 100%; height: 100%;
  background: rgba(0, 0, 0, 0.6);
  display: flex; align-items: center; justify-content: center;
  z-index: 1000;
}

.modal-content {
  background: #fff;
  width: 90%; max-width: 600px;
  padding: 24px;
  border-radius: var(--radius);
  position: relative;
  box-shadow: 0 10px 30px rgba(0,0,0,0.3);
  animation: toast-in 0.3s ease;
}
```

4. Application Architecture and Business Logic (app.js)

The **app.js** file contains the core business logic of the system. It is responsible for implementing algorithms, API communication, state management, and dynamic user interface updates.

4.1 Geospatial Services Integration

The application uses the **Nominatim API** to locate cities. After obtaining city coordinates, a request is sent to the **Geoapify Places API** to retrieve nearby tourist attractions. The data retrieval process follows four stages:

1. City geocoding.
2. Place discovery.
3. Additional information retrieval.
4. Map visualization.

Example from our code:

```
// 2. GEOCODING: If no cache, request coordinates from Nominatim API
document.getElementById("results").innerHTML =
  "<p class='placeholder'>Searching for " + cityName + "...</p>";

const url =
  "https://nominatim.openstreetmap.org/search" +
  "?city=" + encodeURIComponent(cityName) +
  "&format=json&limit=1";

fetch(url)
  .then(function (response) { return response.json(); })
  .then(function (data) {
    if (data.length === 0) {
      showToast('City "' + cityName + '" not found. Try another name.',
        "error");
      document.getElementById("results").innerHTML =
        "<p class='placeholder'>No results. Try a different city name.</p>";
      return;
    }
    const lat = data[0].lat;
    const lon = data[0].lon;
    getPlaces(lat, lon, cityName);
  })
})
```

4.2 Request Optimization and Caching

One of the most important project features is the caching system. Before making a new API request, the application checks LocalStorage. If the city was previously searched, the stored results are loaded instantly without contacting the external API again.

Advantages of this approach include:

- improved performance;
- reduced API usage;

- lower network traffic;
- protection against API rate limits.

Example from our code:

```
const cacheKey = "cache_" + cityName.toLowerCase().trim();
const cachedData = localStorage.getItem(cacheKey);
```

4.3 Distance Calculation Using the Haversine Formula

To determine the precise distance between the city center and nearby attractions, the Haversine formula is used. Calculations are performed directly in the browser using JavaScript mathematical functions:

- `Math.sin()`;
- `Math.cos()`;
- `Math.atan2()`;
- `Math.sqrt()`.

The resulting distance values are used for sorting attractions by proximity.

Example from our code:

```
function calculateDistance(lat1, lon1, lat2, lon2) {
  const R = 6371;
  const dLat = (lat2 - lat1) * (Math.PI / 180);
  const dLon = (lon2 - lon1) * (Math.PI / 180);
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(lat1 * (Math.PI / 180)) * Math.cos(lat2 * (Math.PI / 180)) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  const distanceKm = R * c;
  return Math.round(distanceKm * 1000); // Return result in meters
}
```

4.4 Dynamic Loading of Descriptions and Images

Additional content is automatically retrieved using the Wikipedia API.

When a user opens location details, the system:

- determines the coordinates;
- finds the nearest Wikipedia article;
- retrieves a short description;
- loads an image;
- displays all information in the modal window.

This approach provides users with useful contextual information without leaving the application.

4.5 Interactive Map with Leaflet.js

The project integrates the Leaflet.js library for map visualization. The map is based on OpenStreetMap data. When a location card is opened, the application automatically:

- creates a map instance;
- sets the zoom level;
- places a marker;
- displays the selected location.

Additionally, users can open the same location directly in Google Maps using a dynamically generated link.

4.6 Dynamic DOM Manipulation

Search results are rendered dynamically through JavaScript-generated HTML. Advantages of this approach include:

- no page reloads;
- fast interface updates;
- compliance with SPA principles.

Example from our code:

```
const geoUrl =
`https://en.wikipedia.org/w/api.php?action=query&format=json&origin=*&list=geosea
rch&gscoord=${lat}|${lon}&gsradius=300&gslimit=1`;
const geoResponse = await fetch(geoUrl);
const geoData = await geoResponse.json();

const wikiUrl =
`https://en.wikipedia.org/w/api.php?action=query&format=json&origin=*&prop=extrac
ts|pageimages&exintro&explaintext&pithumbsize=600&titles=${encodeURIComponent(sea
rchTitle)}`;
const response = await fetch(wikiUrl);
const data = await response.json();
```

4.7 Filtering and Sorting

A multi-factor filtering system is implemented to improve usability.

Available categories include:

- Tourism;
- Food;
- Entertainment.

Sorting options include:

- distance;
- name;
- rating.

Example from our code:

```
let filtered = [...currentPlaces];

const catSelect = document.getElementById("categoryFilter").value;
const sortSelect = document.getElementById("sortFilter").value;

// 1. FILTERING: Retain only places that match the selected category
if (catSelect !== "all") {
  filtered = filtered.filter(function(place) {
    if (!place.properties.categories) return false;
    return place.properties.categories.some(function(c) {
      return c.includes(catSelect);
    });
  });
}
```

```

    });
  });
}

// 2. SORTING: Reorder the array based on user preference
if (sortSelect === "distance") {
  filtered.sort((a, b) => (a.properties.distance || 999999) -
(b.properties.distance || 999999));
} else if (sortSelect === "name") {
  // Alphabetical sort using localeCompare for accurate string comparison
  filtered.sort((a, b) => (a.properties.name ||
"".localeCompare(b.properties.name || "")));
} else if (sortSelect === "rating") {
  // Sort by rating (highest first). Fallback to 0 if rating is missing
  filtered.sort((a, b) => (b.properties.rating || 0) - (a.properties.rating ||
0));
}

renderResults(filtered);
}

// SORTING TRIP LIST
function sortTrip(type) {
  if (trip.length === 0) {
    showToast("Your list is empty!", "info");
    return;
  }

  if (type === 'name') {
    // Sorted by name (from A to Z)
    trip.sort(function(a, b) {
      return a.name.localeCompare(b.name);
    });
  } else if (type === 'category') {
    // Sorted by category (from A to Z)
    trip.sort(function(a, b) {
      return a.category.localeCompare(b.category);
    });
  }

  saveTrip(); // Save sorted list
  renderTrip();
  showToast("Sorted by " + type, "success");
}

```

The application follows the principle of **Data Immutability** by creating a copy of the original array before applying sorting operations.

4.8 Travel List State Management

The Trip List functions as an independent subsystem of the application.

Users can:

- add locations;
- remove locations;
- sort saved entries;
- create custom entries.

All modifications are automatically stored in LocalStorage.

Example from our code:

```
let trip = JSON.parse(localStorage.getItem("trip")) || [];  
  
function saveTrip() {  
  localStorage.setItem("trip", JSON.stringify(trip));  
}  
  
function renderTrip() {  
  const list = document.getElementById("tripItems");  
  const emptyMsg = document.getElementById("emptyMsg");  
  
  emptyMsg.style.display = trip.length === 0 ? "block" : "none";  
  
  list.innerHTML = trip  
    .map(function (place, index) {  
      return (  
        '<li>' +  
        '<span>' + place.name +  
        ' <small style="color:#6b7280">(' + place.category + ')</small>' +  
        '</span>' +  
        '<button class="remove-btn" onclick="removeFromTrip(' + index +  
        ')>X</button>' +  
        '</li>' +  
      );  
    })  
    .join("");  
}
```

As a result, user data remains available even after refreshing the page.

4.9 Duplicate Data Protection

Before adding a location to the travel list, the application verifies whether the place already exists. This validation is performed using JavaScript's `some()` method.

Example from our code:

```
function addToTrip(name, category) {
  if (trip.some(function (p) { return p.name === name; })) {
    showToast('"' + name + '" is already in your list!', "info");
    return;
  }
  trip.push({ name: name, category: category });
  saveTrip();
  renderTrip();
  showToast('"' + name + '" added to your trip!', "success");
}
```

As a result, duplicate entries cannot be added accidentally.

4.10 Input Validation and Error Handling

To ensure stability and reliability, several validation mechanisms are implemented.

The application checks:

- empty input fields;
- excessively short names;
- network errors;
- missing search results.

Example from our code:

```
document.getElementById("customPlaceForm").addEventListener("submit", function(e)
{
  e.preventDefault();
  const nameInput = document.getElementById("customName");
  const catInput = document.getElementById("customCategory");

  const name = nameInput.value.trim();
  const category = catInput.value;
```

```

    if (name.length < 2) {
        showToast("Name must be at least 2 characters long.", "error");
        return;
    }
    addToTrip(name, category);
    nameInput.value = "";
});

document.getElementById("searchBtn").addEventListener("click", function () {
    const city = document.getElementById("searchInput").value.trim();
    if (city === "") {
        showToast("Please type a city name first.", "info");
        return;
    }
    searchCity(city);
});

document.getElementById("searchInput").addEventListener("keydown", function (e) {
    if (e.key === "Enter") document.getElementById("searchBtn").click();
});

.catch(function () {
    showToast("Network error. Check your internet connection.", "error");
    document.getElementById("results").innerHTML =
        "<p class='placeholder'>Could not connect. Try again.</p>";
});

```

Whenever an error occurs, the user receives an appropriate notification through the Toast Notification system.

4.11 Unit Testing

Automated unit tests were implemented to verify the correctness of key algorithms. The tests validate:

- duplicate protection;
- alphabetical sorting;
- distance sorting.

Testing is executed automatically after page initialization, and the results are displayed in the browser console.

Example from our code:

```

function runUnitTests() {
  console.log("%cRUNNING TRAVEL PLANNER TESTS", "color: blue; font-weight: bold; font-size: 14px;");

  const originalTrip = [...trip];

  try {
    // TEST 1: Duplicate protection
    trip = [];
    addToTrip("Test Rome", "tourism");
    addToTrip("Test Rome", "tourism");

    if (expect(trip.length).toBe(1)) {
      console.log("%cTest 1 (Duplicate protection) passed successfully.", "color: green");
    }

    // TEST 2: Alphabetical sorting
    trip = [
      { name: "Zebra Park", category: "park" },
      { name: "Apple Store", category: "shop" }
    ];
    sortTrip("name");

    if (expect(trip[0].name).toBe("Apple Store")) {
      console.log("%cTest 2 (Alphabetical sorting) passed successfully.", "color: green");
    }

    // TEST 3: Distance sorting
    let mockPlaces = [
      { properties: { name: "Far Place", distance: 1000 } },
      { properties: { name: "Nearest Place", distance: 150 } },
      { properties: { name: "Mid Place", distance: 500 } }
    ];

    mockPlaces.sort(function(a, b) {
      const distA = a.properties.distance || 999999;
      const distB = b.properties.distance || 999999;
      return distA - distB;
    });

    if (expect(mockPlaces[0].properties.name).toBe("Nearest Place")) {
      console.log("%cTest 3 (Distance sorting) passed successfully.", "color: green");
    }

  } catch (error) {
    console.error("Critical error during testing:", error);
  }

  trip = originalTrip;
}

```

```
saveTrip();
renderTrip();

console.log("%cTESTS COMPLETED", "color: blue; font-weight: bold;");
}
setTimeout(runUnitTests, 2000);

function closeModal() {
  document.getElementById('detailsModal').style.display = 'none';
}
```

This enables quick verification of application functionality without requiring a backend environment.

5. Conclusions

As a result of this project, a fully functional Travel Planner Application was successfully developed, demonstrating the core principles of modern client-side web development.

During implementation, several external services and libraries were integrated, including Nominatim API, Geoapify API, Wikipedia API, OpenStreetMap, and Leaflet.js. The project also incorporates LocalStorage-based caching, travel list management, interactive maps, automatic retrieval of descriptions and images, advanced filtering and sorting mechanisms, and automated unit testing.

A distinctive feature of the application is the custom implementation of the Haversine formula for distance calculation, the use of Data Immutability principles, the creation of a custom SVG logo, and the adoption of the Single Page Application architecture.

The final product is a scalable, user-friendly, and feature-rich web application that demonstrates practical skills in modern web technologies, API integration, client-side architecture, and data-processing algorithms.